

PROJECT REPORT

Personal Portfolio & Real-Time Expense Management

Technical Specification & Engineering System Report

<u>Submitted by:</u>	Abhinav Singh
<u>Registration No.:</u>	25BAI10303
<u>Course:</u>	Programming in Java
<u>University:</u>	VIT Bhopal University
<u>Date of Submission:</u>	18 September 2026

EXECUTIVE SUMMARY & SYSTEM SCOPE

Modern university students and early-career working professionals encounter systemic friction in tracking personal finances. Living outflows are heavily fragmented across digital banking channels, credit accounts, and digital wallet networks, whereas long-term investments are confined to isolated brokerage applications. Manual tracking via spreadsheets introduces human data-entry errors and a lack of live market asset valuation.

Personal Portfolio & Real-Time Expense Management resolves this inefficiency by uniting daily cash outflows with long-term asset accumulation into a single financial command center. The system provides a dual-layer implementation: an offline-first Core Java SE 17+ / JavaFX 21+ desktop engine using embedded SQLite persistence via JDBC with thread-safe asynchronous API polling (CompletableFuture), Jackson JSON parsing, and reactive visualization—alongside an interactive React 19, TypeScript, and Tailwind CSS web platform simulator.

1. Introduction

Personal Portfolio & Real-Time Expense Management bridges short-term living expenditure with long-term net worth growth. Personal finance tools often fall into two opposite extremes: simple expense trackers that ignore investment growth, or complex trading dashboards that omit daily budget enforcement. This system unifies both disciplines into a single desktop and web application.

2. Problem Statement

- **Fragmented Financial Ecosystems:** Outflows occur across checking accounts and credit cards, while holdings are trapped in separate brokerage apps.
- **Spreadsheet Fragility & Overhead:** Manual tracking suffers from human error, high upkeep effort, broken formulas, and zero market integration.

- **Absence of Proactive Budget Guardrails:** Standard accounting tools retroactively record spending without alerting users before budget caps are breached.
- **Unrealized Asset Valuation Gaps:** Net worth calculations often omit real-time price fluctuations, resulting in delayed visibility into actual wealth.

3. Functional Requirements

Req ID	Functional Module	Detailed Requirement & Operational Behavior
FR-01	KPI Command Center	Compute and display total Net Worth, available liquid cash in INR (₹), month-to-date spending, and unrealized portfolio P&L;
FR-02	CRUD Transaction Ledger	Support full CRUD for Income, Expense, Asset Buy, and Asset Sell entries with input validation (amount > 0.0, category, account).
FR-03	Budget Alert Engine	Maintain spending limits across categories. Trigger visual statuses: SAFE (<80%), WARNING (80%–99%), and CRITICAL (>=100%).
FR-04	Live Market Pipeline	Fetch market quotes for equities and ETFs asynchronously without blocking the UI thread using Jackson deserialization.
FR-05	Visual Analytics	Render SVG donut charts for category distributions alongside monthly financial health reports showing gross inflows and savings rates.
FR-06	Embedded Persistence	Store ledgers, category limits, and holdings locally in SQLite via parameterized JDBC statements and HikariCP pooling.

4. Non-Functional Requirements

Quality Attribute	Target Benchmark Metric	Architectural Specification
Performance	60 FPS UI / <10ms Query Latency	Completable Future threads offload REST calls from UI thread; HikariCP optimizes connection reuse.
Data Isolation	100% Local Storage / Offline First	Zero mandatory external cloud dependencies; financial ledgers remain isolated inside an embedded SQLite DB file.
Security	Zero SQL Injection Risk	All persistence operations rely exclusively on parameterized JDBC Prepared Statement interfaces.
Reliability	ACID Transaction Compliance	Database updates execute within explicit transaction boundaries with rollback handlers.
Maintainability	Decoupled Codebase & Type Safety	Strict MVC layer isolation in Java backend, complemented by explicit domain interfaces in TypeScript 5+.

5. System Architecture

The system follows a Model-View-Controller (MVC) architecture to ensure clear separation of concerns:

PRESENTATION LAYER (VIEW)

JavaFX 21+ Desktop Engine | React 19 Web App (TableView Ledger, Donut Charts, Live Market Ticker, Alert Badges)

↓ *Events, Observable Collections & State Observers* ↓

CONTROLLER & BUSINESS ENGINE LAYER

DashboardController.java | TransactionController.java | BudgetAlertEngine.java | MarketDataService.java

↓ *Thread-Safe CRUD Operations & Thread Boundary Sync* ↓

MODEL & DATA PERSISTENCE LAYER

TransactionModel.java | AssetHolding.java | DatabaseManager.java (SQLite 3 via HikariCP JDBC Pool)

6. Schema & Relational Design

Relational Entity-Relationship (ER) Schema Specifications:

```
-- 1. Ledger Transactions Entity Table
CREATE TABLE transactions (
  id TEXT PRIMARY KEY,
  type TEXT NOT NULL CHECK (type IN ('INCOME', 'EXPENSE', 'ASSET_BUY', 'ASSET_SELL')),
  category TEXT NOT NULL,
  amount REAL NOT NULL CHECK (amount > 0.0),
  date TEXT NOT NULL,
  description TEXT NOT NULL,
  account TEXT NOT NULL
);

-- 2. Category Budget Caps Entity Table
CREATE TABLE budget_limits (
  category TEXT PRIMARY KEY,
  monthly_limit REAL NOT NULL CHECK (monthly_limit >= 0.0)
);

-- 3. Multi-Asset Portfolio Holdings Entity Table
CREATE TABLE portfolio_holdings (
  symbol TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('STOCK', 'CRYPTO', 'ETF')),
  shares REAL NOT NULL CHECK (shares >= 0.0),
  average_buy_price REAL NOT NULL CHECK (average_buy_price >= 0.0), current
  price REAL NOT NULL CHECK (current_price >= 0.0)
);
```

7. Design Decisions & Technical Rationale

Vector	Selected Tech	Engineering Justification
Embedded DB	SQLite 3 via JDBC	Eliminates external server management while maintaining full ACID compliance and a lightweight footprint.
Connection Pool	HikariCP	Prevents SQLite thread locks by managing high-performance connection pooling across background workers.
Threading	Java CompletableFuture	Prevents UI freezes during REST API network calls by handling HTTP requests asynchronously.
JSON Parsing	Jackson Databind	Enables fast, strongly-typed mapping between raw REST feeds and domain POJOs.
Web Simulator	React 19 + TypeScript 5	Delivers an accessible web platform that mirrors Java desktop state mechanics in modern browsers.

8. Implementation Details & Core Source Code

1. Budget Alert Engine Logic (Java Core):

```
public class BudgetAlertEngine {
    private static final double WARNING_THRESHOLD = 0.80; // 80% Spent
    private static final double CRITICAL_THRESHOLD = 1.00; // 100% Limit Breach

    public enum BudgetStatus { SAFE, WARNING_THRESHOLD, CRITICAL_EXCEEDED }

    public BudgetStatus evaluateCategory(double currentSpent, double monthlyLimit) {
        if (monthlyLimit <= 0.0) return BudgetStatus.SAFE;
        double utilization = currentSpent / monthlyLimit;
        if (utilization >= CRITICAL_THRESHOLD) return BudgetStatus.CRITICAL_EXCEEDED;
        if (utilization >= WARNING_THRESHOLD) return BudgetStatus.WARNING_THRESHOLD;
        return BudgetStatus.SAFE;
    }
}
```

2. Thread-Safe Transaction Insertion (JDBC PreparedStatement):

```
public void insertTransaction(TransactionModel tx) throws SQLException {
    String sql = "INSERT INTO transactions (id, type, category, amount, date, description, acc
ount) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (Connection conn = HikariDataSourceManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, tx.getId());
        pstmt.setString(2, tx.getType().name());
        pstmt.setString(3, tx.getCategory());
        pstmt.setDouble(4, tx.getAmount());
        pstmt.setString(5, tx.getDate().toString());
        pstmt.setString(6, tx.getDescription());
        pstmt.setString(7, tx.getAccount());
        pstmt.executeUpdate();
    }
}
```

9. Verification Results & Visual Simulation

COMMAND CENTER DASHBOARD VISUAL LAYOUT SIMULATION

NET WORTH: ■12,45,800.00 | **LIQUID CASH:** ■1,85,200.00 | **PORTFOLIO P&L::** +■1,12,400 (+14.2%)

MONTHLY EXPENSE LEDGER:

- Rent & Housing: ■25,000.00 (Rent)
- Dining: ■1,200.00 (Food & Dining)
- Equities: ■15,000.00 (ASSET_BUY)

BUDGET STATUS CAPS:

- Groceries: ■6,500 / ■10,000 [SAFE]
- Utilities: ■4,800 / ■5,000 [WARNING]
- Entertainment: ■8,200 / ■6,000 [CRITICAL EXCEEDED]

10. Testing Approach & Suite Results

Test Suite Class	Target Test Method	Verification Assertion	Status
TransactionValidationTest	shouldRejectNegativeAmount()	Asserts exception handling when transaction amount is <= ■0.00.	PASSED
TransactionValidationTest	shouldRejectEmptyDescription()	Asserts validation failure on null or empty whitespace strings.	PASSED
DatabaseManagerTest	shouldPersistAndRetrieve()	Verifies SQLite PreparedStatements insert and select operations.	PASSED
BudgetAlertEngineTest	shouldTriggerWarningAtEighty()	Asserts WARNING_THRESHOLD status at 85% budget consumption.	PASSED
BudgetAlertEngineTest	shouldTriggerCriticalExceeded()	Asserts CRITICAL_EXCEEDED status when expenses breach 100% cap.	PASSED
JacksonParserTest	shouldDeserializeMarketQuote(s)	Asserts Jackson ObjectMapper maps REST JSON payload to POJO.	PASSED

11. Challenges Faced & Technical Mitigations

- **JavaFX UI Thread Blocking:** Ingesting REST API updates synchronously caused temporary UI freezes. *Mitigation:* Replaced synchronous calls with CompletableFuture worker threads, using Platform.runLater() for UI updates.
- **SQLite File Locks in Multithreaded Contexts:** Concurrent background writes resulted in database lock errors. *Mitigation:* Integrated HikariCP connection pooling and enabled Write-Ahead Logging (WAL) mode.
- **Cross-Platform Model Synchronization:** Keeping TypeScript models aligned with Java POJOs required consistent property mapping. *Mitigation:* Unified structural schemas across both implementations.

12. Learnings & Key Takeaways

- **Asynchronous Concurrency Models:** Offloading I/O and network operations away from the main UI thread is critical for maintaining high rendering responsiveness (60 FPS).
- **Defensive Storage Management:** Parameterized SQL queries combined with HikariCP connection pooling prevent both security vulnerabilities and multithreaded resource lock contention.
- **Modular Decoupling:** Implementing strict MVC separation allows UI presentation layers (JavaFX / React) to evolve independently of core calculation engines.

13. Future Enhancements & System Roadmap

- **Automated SMS & Email Receipt Parsing:** Regex and NLP ingestion to parse incoming bank transaction SMS messages directly into ledger entries.
- **Predictive ML Cash Flow Forecasting:** Time-series analysis models (ARIMA / Prophet) to forecast month-end cash balances and warn of projected shortfalls.
- **AES-256 Storage Encryption:** Integration of SQLCipher for full disk encryption of local financial database files.

14. References

[1] OpenJFX Consortium. *JavaFX 21 Architecture & User Interface Documentation*. <https://openjfx.io/>

[2] React Documentation. *React 19 State Architecture and Server Components*. <https://react.dev/>

- [3] SQLite Development Team. *SQLite 3 In-Memory and Embedded Storage Mechanics*.
<https://www.sqlite.org/docs.html>
- [4] HikariCP Engineering. *High-Performance JDBC Connection Pooling Optimizations*.
<https://github.com/brettwooldridge/HikariCP>
- [5] Jackson Databind Guide. *High-Throughput JSON Data Binding in Java*.
<https://github.com/FasterXML/jackson-databind>